CODE & SYSTEMS

L12 — Build a small persistent subsystem

Current guide Audited 9 November 2026 GUIDE-1 · c166ec650b98

Study the implemented vessel system, then make a neutral practice survey: start work at one position, wait three paid phases, inspect freely, cancel or move away, collect a result and preserve its count after leaving the scene and restarting. This is an unfamiliar state machine with real input and save integration, not a single extra JSON flag.

First study the production vessel path

Definition and identity. `vessels.py` validates versioned configuration and creates an exact physical UUID plus capacity/water. Two same-ID pots are distinct owners. Water weight contributes to carrying. Modern intentional absence must not become legacy clay-pot initialization during reconstruction.

Selection and commitment. `vessel_menu.py` captures an ordinary choice but rechecks current ownership, water, requirements and context when the action commits. A stale selected pot is not replaced by a same-ID sibling. `vessel_jobs.start` revalidates materials, vessel, heat/reach and output, then transfers the exact vessel and committed work into `LocalMap.processing_layer`.

Progress and failure. Engine processing advances from paid phases, not render frames. Output construction/delivery is preflighted; `vessel_jobs.deliver` can roll back partial placement. Unfinished work is retained if output cannot be built. `abandon` returns the empty physical vessel and discards committed ingredients. Cancellation before commitment and abandonment after commitment are different policies.

Persistence. `save_components.serialize_item/validate_item/restore_item` and map/job validation preserve quantity, UUID, owned work and compatibility rules. A cold process reconstructs registered code and objects before continuing. SYS-1 delivered this system; SYS-2 delivered shared light. This source study is not a new execution of every vessel scenario. L04 executes the simpler passive-work boundary; the current vessel source/authoritative guide provides the complete production case.

Independent survey starting point

The full patch, three tool definitions, perk definition and solution are supplied. The fixture grants the purchase prerequisite, then acquires the perk through K and uses tools through inventory. Predict whether the starting action counts as phase one, whether inspection ages work, and whether malformed saved state should partially replace the current player.

POWERSHELL

```
& $ResonancePython -X utf8 -B Resonance_Wiki_Maintenance/practice.py L12 --output evidence/guide-1/l12
```

```text
absent --use survey / pay 1--> working(left=2)
working --wait in place / pay 1--> working(left=1) --> ready
working --move or scene change--> idle (work lost)
working --cancel / free--> idle (work lost)
working --use again / rejected free--> working
any --inspect / free--> same state + message
ready --use survey / pay 1--> idle, completed += 1
```

Name the owner and schema

The owner is `player.perks.perk_state['guide1_survey']`; configuration is `SURVEY_PHASES = 3` in the lesson module. The full record is `{version, phase, left, anchor, completed}`. The anchor captures world coordinates, submap ID and local position while working. Idle/ready require zero work and no anchor. Working requires positive bounded work and a valid anchor. Counters are exact nonnegative integers, excluding booleans; version must be supported.

Absent state is an explicit compatible default, initialized lazily. Present incomplete/malformed state is refused. The existing outer dictionary serializer is reused, but its general object validation is insufficient; `validate_states` supplies inner rules. A `version` label alone does not validate anything.

Connect actions, time and feedback

`use_survey` requires acquisition, rejects reentry while working, starts or collects work and sets commitment. `use_inspect` logs without commitment; `use_cancel` abandons unfinished work freely by this lesson's explicit policy. `survey_turn` consumes the real ON_TURN event, compares the captured anchor and advances once. The starting inventory action fires ON_TURN immediately, so three becomes two before the next input.

The handler returns messages through the real effect merger/consumer. Duplicate registry rebuilds use the same tagged-handler mechanism as L11. The complete definitions and every function are embedded in Full code and practice solutions. The installed patch is active in all tested processes, including cold restore; no final implementation step is omitted.

Prove persistence rather than merely writing JSON

The scenario saves in the working phase, checks same-process equality, starts another process with the same patch, verifies the snapshot and **finishes/collects** the saved work there. The parent independently finishes, tests reentrant rejection and free inspection/cancel, moves away to cancel work, leaves/reenters the scene and cold-loads the collected count. Detached deep-copied malformed records are checked by production preflight, then published with valid checksums into separate disposable stores: actual `SaveManager.load` refuses them without replacing live state. Another isolated checkpoint omits the lesson entry and loads to the idle default; the original good checkpoint is restored afterward. Receipts distinguish these checks from L10's original-loop demonstration.

For a wholly new component owner, additionally add component construction/attachment, explicit serializer field/block, strict validator, absent-version default/migration, reconstruction, scene/player transfer semantics and lifecycle registration. Decide whether inactive scenes freeze or advance it. Validate before replacing live objects, restore derived effects without replaying time, and test owner death/removal/transfer. Reusing Perks here avoids those extra owner changes; it does not imply arbitrary attributes save automatically.

Vary, diagnose, make

Guided variation: require four phases. Change `SURVEY_PHASES` to 4, regenerate the reviewed patch contract, update expected initial remaining work to 3 and cold continuation accordingly. Version 1 counters captured under an earlier duration need a deliberate compatibility decision: allowing remaining values up to the new bound is compatible for an increase, while shrinking the bound can reject existing work. Document it.

Fault: restore the outer dictionary but forget inner validation, or install only `save_components.validate_components` after SaveManager imported its alias. A boolean or working-with-zero record can then reach callbacks. Hook dispatch catches errors, so failure may look like stalled work. Test the loader's actual alias before changing the ticker.

Independent task: implement a short unfamiliar task with at least three states, an inspection route, cancellation/failure policy and a later consequence. Choose the owner deliberately. Acceptance includes ordinary input, exact first tick, no render tick, repeated-registration safety, absent/malformed distinction, same/fresh-process continuation and scene leave/reentry. A renamed flag is insufficient.

Hint 1 — Write valid states before code

List which combinations of phase, remaining work and owner/location are legal. Reject everything else before replacement.

Hint 2 — Relevant source

Read `solutions.py:survey_state/use_survey/survey_turn/validate_states/install`, `save_components.validate_components` and `SaveManager._prepare`.

Hint 3 — Partial step

Install registrations before loading. Save with two phases left; a fresh process must load, pay those phases, then collect through inventory.

Complete solution and rationale

The delivered state machine and fixed patch are the full answer. It starts at three, counts the starting phase, inspects/cancels freely, loses work on movement, retains completed history and refuses malformed presence. The four-phase variation changes configuration/expected evidence with an explicit compatibility policy. Valid independent alternatives include staged study or calibration, provided they have meaningful transitions and an observable saved consequence. Neither must become new production canon.

Lifetime and restart

Running functions/configuration are process-local. New actions use installed configuration; loaded player state is validated/reconstructed. Player-owned completed history crosses scenes; working location is checked on the next paid phase. New worlds default absent. Rerun the fixture for a clean state. If manually installing a patch, follow the recovery manifest procedure and

never blanket-reset unrelated work. Revisit after a delay by designing malformed examples that a dictionary-only validator would wrongly accept.